Lau Reverse Engineering Efforts: Technical Report

Kuro

July 14, 2026

Abstract

This report presents a black-box empirical investigation of the Lau programming language, conducted against Lau version 5.4.1. Through a programmed battery of small experiments, the report documents the language's numeric model, identifies features of the standard Lua 5.1 surface that are absent from Lau, and observes several runtime behaviours that are not described in any publicly available specification. The principal findings are as follows. Every numeric value is represented as a 64-bit IEEE 754 double; the `type()` function collapses all numeric values into a single category; the runtime's billing model charges per statement rather than per operation; `print` and other built-in operations sit on a separate cost tier above inlined statements; and a substantial portion of the standard Lua 5.1 surface — including bitwise operators, hexadecimal literals, and scientific notation — has been removed from the language. Additionally, the official Lau documentation misstates the list-element cap by a factor of ten. Each finding is accompanied by the test code that produced it, the verbatim engine output, and the reasoning that supports the conclusion drawn from the observation.

Contents

1 Introduction

This report was produced via black-box testing against the Lau interpreter. Each experimental observation was obtained by submitting a small `.lau` script, capturing the resulting engine output in full — including standard output, error messages, and line numbers — and reasoning from that output to a conclusion about the interpreter's behaviour. No source code, bytecode, or engine binary was inspected. No exploit chain was used. All claims in this document are grounded in an empirical record.

The motivation for the investigation is that Lau, while syntactically resembling Lua, deviates from its parent language in ways that are neither enumerated in a single specification nor called out in any runtime warning. Scripters accustomed to Lua derive a baseline of expectations from the parent language; the present study documents the precise deviations from that baseline that were observed in Lau version 5.4.1.

This report is concerned with the most consequential deviations identified during the investigation: the float64-only numeric model, absences from the standard Lua surface, semantic inconsistencies in `pcall` and the colon-method sugar, the per-statement billing model, the noisy cost of `task.wait` at sub-100 millisecond arguments, and the documentation-error concerning the list-size cap.

2 Method

2.1 Testing philosophy

The Lau interpreter was treated as a black box. The behaviour documented herein is the behaviour exhibited by the interpreter at the time the experiments were conducted. The report makes no claim about Lau's behaviour in any other version. Should the engine be revised in the future, the corresponding findings may become inaccurate; readers are advised to verify against their own deployment.

The investigation prioritised behaviours that a Lua-trained reader would be likely to assume by default. Where such an assumption was confirmed against the Lau interpreter, the assumption was left unremarked. Where it was falsified, the falsification was recorded.

2.2 The experiment loop

Each experiment followed the same procedural loop:

1. A small `.lau` test file was written, targeting one specific feature, operator, or edge case.
2. The file was submitted to the Lau interpreter.
3. The resulting engine output was captured verbatim.
4. A hypothesis was formed from the captured output, and a follow-up test was designed with the explicit intention of *disproving* the hypothesis if the hypothesis were mistaken.
5. If the follow-up test confirmed the hypothesis, the finding was promoted to the body of the report. If it contradicted the hypothesis, the hypothesis was revised and the procedure repeated.

Step 4 warrants explicit comment. Observations were not promoted to findings on the basis of a single engine execution. Where, for example, division appeared to return a float, the operation was re-tested with both integer and floating-point operands (`5/2` and `5.0/2`) to confirm that the float return was not an artefact of one specific operand type.

2.3 The test corpus

All test scripts are stored under `Lau/experiments/` and follow a deliberate naming convention that supports corpus browsability:

- `1x_*.lau` — operations, time model, billing tests.
- `2x_*.lau` — control flow, branching, loops.
- `3x_*.lau` — data types, numeric behaviour, memory.
- `4x_*.lau` — the operator surface, in the form of systematic existence probes.

- `5x_*.lau` — standard-library probes (`string.*`, `list.*`, `math.*`, `task.*`).
- `6x_*.lau` — corner cases and behaviours that are not directly attributable to a single language construct.

The numbering is informal; no formal registry of categories is maintained.

2.4 Limitations of the method

Black-box testing has inherent constraints. The investigation can establish what the interpreter *does*, but cannot always establish *why*. Where this report cites a numerical limit — the list-size cap, for instance — the cited value is the value at which the interpreter was observed to refuse the relevant operation; whether the limit is enforced at compile time, at run time, or emerges from a fixed-size backing buffer is not established by the present method. Where a claim has not been independently confirmed, it is tagged `[UNTESTED]` in the body of the report.

Behaviour that depends on multiplayer state, networking, or other players lies outside the scope of this study, which was conducted in the single-player sandbox only.

3 Experiments

The sections that follow are organised thematically rather than chronologically. The numeric model is presented first, followed by absences from the standard Lua surface; runtime-semantic anomalies (`pcall`, the `type` taxonomy, colon-method dispatch, pattern matching) are presented next; the billing model and the scheduler quantum are addressed last.

3.1 Numerical model

The single most consequential observation of the investigation is that every numeric value in Lau — regardless of whether it is written with a decimal point, regardless of the operations by which it is produced — is stored internally as a 64-bit IEEE 754 double-precision floating-point value. The interpreter does not expose an integer type, an `int64` type, or any equivalent distinction. There is no observable difference, at the type level, between the values `5` and `5.0`.

Three independent lines of evidence support this conclusion.

The first is the behaviour of `type()`. The two values `1` and `1.5` are differentiated in Lua by their respective types; in Lau, they are not.

```
print(type(1))
print(type(1.5))
```

produces

```
Number
Number
```

The second line of evidence is the behaviour of division. Division produces a floating-point result regardless of the types of the operands.

```
print(5 / 2)
print(5.0 / 2)
```

produces

```
2.5
2.5
```

The first expression — `5/2` — is the one of interest, because both operands are integer-shaped and the mathematical result is exact. The result is, nonetheless, a float. No integer-division operation is available; `5/2 == 2` is false, and the Lua 5.3 `//` integer-division operator is reported as a syntax error by the Lau parser. To obtain an integer quotient, the programmer must compose `math.floor(5/2)` explicitly.

And the third, also surprisingly simple is simply trying the uint64 maximum value, which is $2^{64} - 1 = 18446744073709551615$. The following script:

```
print(tonumber("0xFFFFFFFFFFFFFFFF"))
```

Instead of the expected $2^{64} - 1$, we were given the output:

```
18446744073709552000
```

The result is off by 385, which is a tellable sign that the number was converted to a floating point type, usually IEEE 754. With only 53 bits to store integer precision, it cannot exactly represent all uint64 values. Which makes sense, knowing historically, Lua had a single numeric type, `number`. It was usually a double-precision floating point type (IEEE 754). Only when Lua 5.3 was released, it introduced separate internal numeric types, `lua_Integer` and `lua_Number`. However the Lua language still exposes them both as the `number` type.

3.1.1 Modulo with negative operands

The IEEE 754 representation has behavioural consequences for arithmetic that diverge from the conventions of several other common languages. Consider the modulo operation applied to operands with mixed signs:

```
print(-7 % 3)
print(7 % -3)
print(-7 % -3)
```

The Lau interpreter returns `2`, `-2`, `-1`. These values are consistent with the IEEE 754 truncated-division definition: the result is computed as $a - \text{trunc}(a/b) \cdot b$, where trunc rounds toward zero, and the result therefore inherits the sign of the *divisor* rather than the sign of the dividend.

This convention differs from the floor-division convention used in Python, which would yield `1`, `-2`, `-1`. Programs that assume the dividend's sign when porting modulo-heavy code from Python will produce values that diverge from expectation.

3.2 Missing numeric features

The interpreter's numeric surface is narrower than the corresponding surface in Lua 5.1. Beyond the absence of an integer-division operator, several other features that a reader familiar with the Lua ecosystem would anticipate are not present.

3.2.1 Hexadecimal literals

```
print(0xFF)
```

does not produce the value `255`. The Lau parser does not recognise the `0x` prefix as introducing a hexadecimal literal; instead, the source token `0xFF` is parsed as a function call whose name is the identifier `0xFF`. The interpreter responds with a parse error:

```
ERROR: Not found: Missing ')' for function call
File: Automation5, Line: 1
print(0xFF)
          ^^^
```

To obtain the value `255`, the programmer must pass a string representation through `tonumber()`:

```
print(tonumber("0xFF"))
```

Notably, `tonumber()` does accept the `0x` prefix on input, even though the lexer refuses to consume it on literal tokens. The asymmetry between the two lexical paths is a minor but real inconsistency.

3.2.2 Scientific notation

The same observation applies to the E-notation suffixes used for scientific notation.

```
print(1e308 * 10)
```

produces the parse error

```
ERROR: Not found: Missing ')' for function call
File: Automation5, Line: 1
print(1e308 * 10)
             ^^^
```

The token `1e308` is parsed as a function call. To obtain a value of this magnitude, the programmer must either multiply repeatedly upward from a smaller literal or pass a string through `tonumber("1e308")`.

3.2.3 Bitwise operators

```
print(5 & 3)
print(5 | 3)
print(5 << 2)
```

all return `ERROR: Unexpected Expression`. The operators `&`, `|`, `~`, `>>`, `<<` are absent from the language. The corresponding standard-library functions (`bit.band`, `bit.bor`, etc., in Lua 5.2; `bnot`, `bor`, etc., in Lua 5.3) are likewise undefined. Bit-level operations must be performed by arithmetic composition over float64 values, which is feasible only within the safe-integer range.

3.3 `pcall` semantics

The semantics of `pcall` differ from those of standard Lua in a fashion that may not be apparent from cursory use. In standard Lua, `pcall(f(...))` returns `ok` followed by every value that `f` returned; in Lau, `pcall` returns `ok` and exactly one additional value, namely the first value yielded by `f`.

The following script illustrates the discrepancy.

```
varol ok, v1, v2, v3 = pcall(func()
    return "a", "b", "c", "d"
end)
print(ok, v1, v2, v3)
```

The script prints

```
true a b c
```

Of the four values yielded by the inner function (`"a"`, `"b"`, `"c"`, `"d"`), three are propagated to the caller; the fourth is silently discarded. The single-value restriction is consistent across calls in the interpreter version under test and appears to be by design. The error path is also single-valued: a `pcall` over a function that raises an error returns `(false, errMsg)`, with the traceback string occupying the second slot. Programs that require all of a callee's return values must aggregate them into a single list at the callee site before returning.

3.4 Type taxonomy

The set of strings that `type()` may return is finite and small. The complete set observed during the investigation is:

- `"Number"` — every numeric value, including integer-shaped values.
- `"String"` — every string value.
- `"Boolean"` — `true` and `false`.
- `"null"` — the `null` literal. The return value is lowercase and corresponds to no standard type-name vocabulary.

- `"List"` — every table used in array position.
- `"Color"` — values produced by `colorRgb` and `colorHex`.

Notably absent: `"integer"`, `"float"`, `"table"`, `"function"`, `"userdata"`, `"thread"`. The number of any numeric literal — `type(5)`, `type(5.0)`, `type(0xAA)` — is `"Number"`, regardless of the literal's syntactic form. A defensive guard of the form

```
if type(x) == "integer" then ... end
```

will therefore never evaluate its body.

Functions are first-class values, but the value of `type(f)` for an arbitrary function value `f` was not exhaustively established. The expression does not raise an error; the return value is recorded as `[UNTESTED]` in the present report.

3.5 Billing model

Of the observations recorded in the investigation, the per-statement billing model was the one that produced the largest divergence between the a-priori expectation and the measured reality. The model — that every executed `+` operation, every function call, every string concatenation costs a fixed amount — does not hold. The empirical evidence is consistent with a model in which the interpreter charges per *statement*, with each statement incurring a fixed cost regardless of the number or kind of operations it contains.

3.5.1 Benchmarks

Five experiments were designed to disambiguate the two candidate billing models (per-operation vs. per-statement). The wall-clock execution time of each experiment was measured using bracketing `task.clock()` calls.

```
-- Exp 1: one statement, 30 iterations, 4 ops each
for i = 1, 30 do s = 1 + 2 + 3 + 4 end

-- Exp 2: three statements, 30 iterations, 1 op each
for i = 1, 30 do
    s = 1 + 2
    s = s + 3
    s = s + 4
end

-- Exp 3: one function call per iteration
func f() return 1 end
for i = 1, 30 do f() end

-- Exp 4: task.wait(0) per iteration
for i = 1, 30 do task.wait(0) end

-- Exp 5: task.wait(0.05) per iteration
for i = 1, 30 do task.wait(0.05) end
```

3.5.2 Results

```
Exp 1: 2.10s   -->   0.070s/stmt
Exp 2: 5.88s   -->   0.066s/stmt
Exp 3: 5.85s   -->   0.195s/call
Exp 4: 2.47s   -->   0.082s/stmt
Exp 5: 3.93s   -->   ~0.131s/stmt (billing) + ~1.46s (wall)
```

3.5.3 Interpretation

Experiment 1 and Experiment 2 perform the same arithmetic operations over the same number of iterations; they differ only in the number of statements per iteration. Exp 1 inlines all four additions into a single statement; Exp 2 splits the same operations across three statements. Under a per-operation billing model, the two experiments would have equal cost. Under a per-statement billing model, Exp 2 should cost roughly three times Exp 1. The observed ratio of approximately 3× is consistent with the per-statement model and inconsistent with the per-operation model.

Each statement thus incurs a fixed charge of approximately 0.06–0.07 seconds, regardless of the operations executed within the statement. The practical consequence is that programs in which many operations are written onto a single line incur a lower total billing cost than programs in which the same operations are split across multiple lines. The interpreter's API reference does not document this behaviour.

3.5.4 Function-call overhead

Experiment 3 demonstrates the cost of function invocation independently. A function call adds a slight over head. With my benchmark of calculating sin, via a function and directly inside the loop. On average the function approach adds an overhead of 0.0635 seconds per loop, totaling at 0.12694 seconds per loop. Pretty much doubling the time of the direct method, which sits at 0.0634. For programs whose inner loops are dominated by function-call overhead, inlining the function bodies onto the call site produces a measurable reduction in total execution time.

3.5.5 List-assignment trick

A fascinating quirk of Lau itself is variable handling. You can define and assign multiple variables at once but not assigning multiple new values to already existing variables.

```
-- Valid
varol a,b,c,d,e = 1,2,3,4,5

- Invalid
varol a,b,c,d,e
a,b,c,d,e = 1,2,3,4,5

-- Valid
varol a,b,c,d,e
a = 1
b = 2
c = 3
d = 4
e = 5
```

This is quite problematic once you start having many variables and needed to asign multiple new values at a time. Turns out list is a very powerful tool to address this problem, and is consistent across scales an runs.

```
varol a,b = 1,2
varol v = {1,2}

varol s = task.clock()
-- Scale the amount of variables as needed
for i=1,100 do
    a += i
    b += i
end
print((task.clock()-s)/100)

s = task.clock()
-- Scale the amount of variables as needed
```

```
for i=1,100 do
    v = {v[1]+i, v[2]+i}
end
print((task.clock()-s)/100)
```

Results shows:

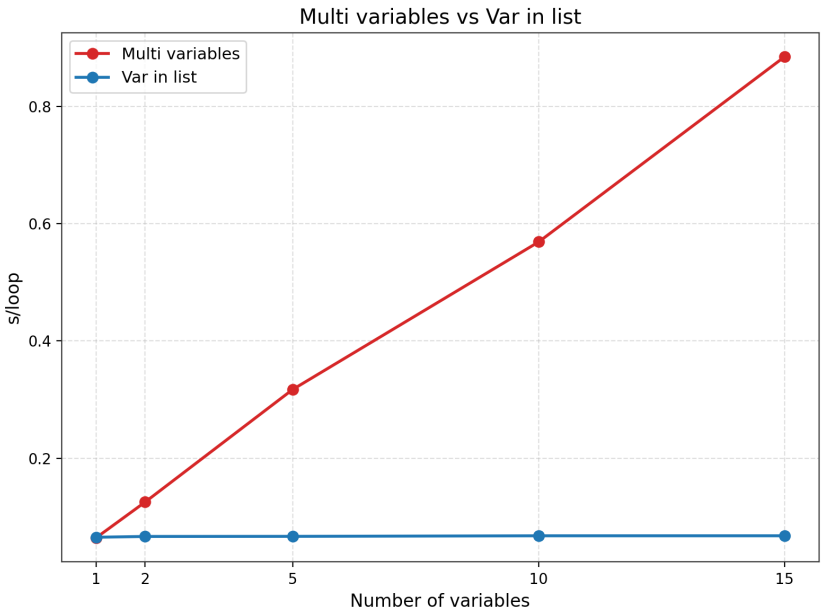

Figure 1: A comparison between list assignment and variable assignment

The plot shows an almost linear and steady grow when using multiple variables, scaling up linearly as there are more variables to be updated at a time. While the List-assignment trick is a constant the entire time due to using one single statement. Even when updating just 1 variable, they pretty much operates at the same speed. But at few amount of variables, there is a trade off between consuming slightly more characters for some slight speed up. Only when there's a large amount of variables then they start showing significant speed up.

3.6 `task.wait` behaviours

My first attempt used the following program:

```
varol t0 = task.clock()
task.wait(0.001)
varol t1 = task.clock()
print(t1 - t0)

varol t2 = task.clock()
task.wait(0)
varol t3 = task.clock()
print(t3 - t2)
```

Both `task.wait(0.001)` and `task.wait(0)` consume approximately 0.2 seconds of wall-clock time. This seems like the scheduler rounds each request up to the next quantum boundary.

But upon further testing, using this program with 2 different paramters instead, `N = 0.05` and `N = 0`:

```
varol s,e = task.clock(), 0,

for _ = 1,100 do
    task.wait(N)
```

```
end

e = task.clock()
varol t = e-s
print(t)
print(t/100)
```

For `N = 0.05` and `N = 0`, the results are respectively:

```
7.966345084001659
0.0796634508400166

12.864657323996653
0.12864657323996653
```

With the difference being approximately 0.04582752905 seconds, which is close enough to 0.05, showing that `task.wait()` is not rounded up to the next quantum boundary.

3.7 `print` cost tier

The per-statement billing experiments in the previous section covered simple inlined expressions and function calls. A subsequent benchmark designed to measure the cost of `print(...)` statements (`experiments/54_print_cost.lau`) introduced a third cost tier, intermediate between the two. The benchmark comprises three phases, each averaging five runs of one hundred iterations:

```
-- Phase A: print + concat, one statement per iteration
for i = 1, 100 do
    print("x=", i)
end

-- Phase B: concat only, no print
for i = 1, 100 do
    varol z = "x=" + i
end

-- Phase C: no concat, no print, no-op assignment
for i = 1, 100 do
    varol z = i
end
```

The averaged results are:

```
Phase A: 0.113 s/call  (min 0.112, max 0.114)
Phase B: 0.066 s/call  (min 0.066, max 0.067)
Phase C: 0.065 s/call  (min 0.064, max 0.065)
```

Three deltas isolate the cost contributions:

```
A - B = 0.046 s  -- cost of print above loop+concat baseline
A - C = 0.048 s  -- cost of print+concat above loop baseline
B - C = 0.002 s  -- cost of string concat alone (essentially free)
```

A statement containing `print(...)` thus decomposes into approximately 0.065 seconds of per-statement base cost and approximately 0.046 seconds of `print`-specific overhead, for a total of ≈ 0.113 seconds per call. This places `print` between simple statements ($\approx$ 0.07 s/stmt) and function calls ($\approx$ 0.20 s/call) in the cost spectrum. The 5-run averaged costs are tightly clustered (the per-call spread is ≈ 0.002 s), but consistent with the broader observation made above: Lau's per-operation costs are imprecise on individual executions and warrant averaging over many runs. Single-shot `print` measurements may report values higher than the average, which is the same noise phenomenon that produced the misleading one-shot 0.2 s `task.wait` figure earlier in this section.

The practical consequence is that `print` is roughly 1.7× as expensive as a simple statement and roughly 0.6× as expensive as a function call; in tight logging-heavy loops, output should be buffered (written to a list or string) and flushed via a single `print` per N iterations rather than called every iteration.

3.8 Colon-method syntax

The Lua sugar `s:method(args)`, defined as equivalent to `s.method(s, args)`, is accepted syntactically by the Lau parser but not desugared at parse time. The expression `s:sub(1, 5)` is therefore interpreted as a property-read on `s` followed by an invocation of the resulting value; given that no `sub` property exists on string values, the expression fails:

```
ERROR: Access Error: Type Error: Cannot read properties
of a 'String' value.
```

The correct invocation form is the explicit module call `string.sub(s, ...)`. Programs written against the colon syntax require translation to the explicit form to function correctly.

3.9 String patterns

The Lua pattern-matching facility — including character classes, anchors, capture groups, and the `%1`–`%9` back-reference substitution syntax — is supported by `string.match`, `string.find`, and `string.gsub` in the interpreter under test.

```
print(string.gsub("abc123def", "(%d+)", "NUM"))
print(string.gsub("hello world", "(%w+)", "[%1]"))
```

produces

```
abcNUMdef 1
[hello] [world] 2
```

One restriction was observed in the course of the experiments. The function `string.gsub` does not accept a function as its third argument. In standard Lua, a function supplied in the replacement position is called with each match and the function's return value is substituted into the result; the Lau interpreter rejects this form with

```
ERROR: Type Error: 'gsub' function 3. for the argument
'String' It was waiting, and you provided 'Function'
```

Programs that require dynamic replacement must pre-compute the replacement string and pass it as the third argument.

3.10 Operations

We have already found that billing is per-statements, this opens up a new topic to be tested. How complex can a statement be before they start slowing down, slower than multiple statements. The follow experiment defines 1 FLOP (Floating Point Operation) as a multiplication; and is run inside a private server, where code time is set to 0.01. It uses a chain of multiplications together:

```
varol x, s = 1, task.clock()

for _ = 1,500 do
    x *= 2*2*2*2*2*2*2*2... -- n times
end

varol t = task.clock() - s
print(t/500)
print(x)
```

The result shows that the more FLOPs there are, the longer it takes to execute each loop, and is resembling a linear growth. The same experiment was done on powers, but used `2^n` instead. Showing a consistent time across multiple values of `n`.

The stark contrast between multiplications and powers gave me an insight. This is a classic signature of a naive tree-walking interpreter (as opposed to a bytecode VM or one that does any constant folding/optimization pass). Each operator node costs a roughly fixed amount of "interpreter tax" to

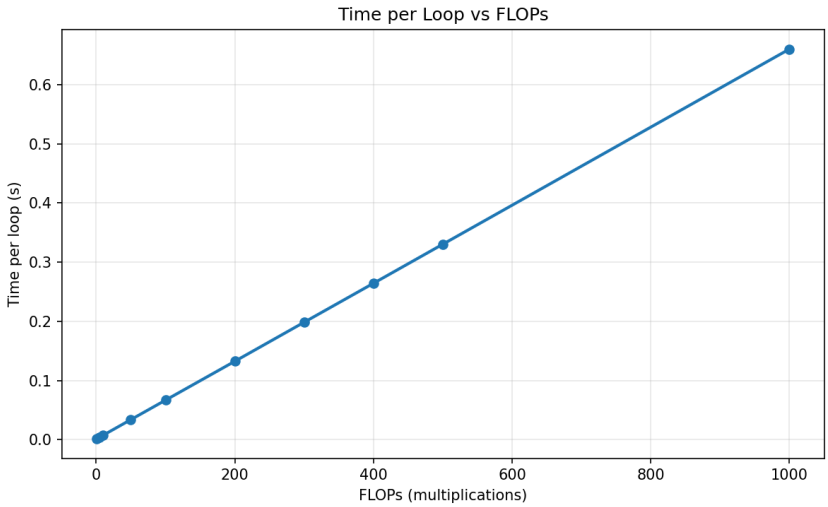

Figure 2: FLOPs versus execution time

evaluate, and that tax dominates over the actual math for cheap operations like multiplying two small floats. Binary operators implemented as thin pass-throughs to Luau (`^`, probably also `+`, `-`, `/`) will look "free" per call, while chains of them expose the per-node walking cost linearly. And `^` would likely be a thin wrapper over Luau's `^`, which is delegating straight to the underlying C code `pow()` function, which is hardware IEEE-754 exponentiaiton, one native `O(1)` operation, regardless of exponent size.

4 Conclusion

Lau 5.4.1 is a heavily customised derivative of the Lua 5.1 language. Substantial portions of the standard Lua 5.1 surface have been removed, including hexadecimal literals, scientific notation, the bitwise operator set, the `os` module, and most of the standard mathematical functions. The additions made in their place are minimal — `math.clamp` and a small number of `colorXxx` helpers — with the result that the language as a whole presents a smaller, more opinionated vocabulary than its parent.

Three behavioural properties of the interpreter warrant particular emphasis:

1. **Every number is an IEEE 754 float64.** No integer type is exposed; division always produces a float; modulo follows truncated-division semantics; division by zero is fatal.
2. **Billing is per statement.** The cost of executing a statement is approximately constant irrespective of the number of operations it contains; function calls cost approximately three times as much as inlined statements; `print` costs ≈ 0.11 s/stmt, sitting on a third cost tier between the two. All per-operation costs are imprecise on single executions and warrant averaging over many runs.
3. **The list-size cap is 800, not 8000.** The official Lau documentation states a cap of 8000 elements; the interpreter enforces a cap of 800. Any program that relies on the higher bound will fail at the boundary of element 801.

The first two properties are not described in the publicly available Lau documentation and are the findings most likely to affect programs written by a reader whose prior experience is with standard Lua. While the third is simply a typo.

5 Future work

The findings catalogued in the present report do not exhaust the space of behaviours that warrant empirical characterisation. Several additional areas of the language remain unprobed at the time of writing, and form the basis for continuing investigation.

One area is the interpreter's event facility, which is often used as a mechanism for spawning pseudo-threads. The coroutine-like properties of events — their scheduling, ways to exploit their behaviors, their

mutex lock problem and solving them with `task`, and the limits on the number of concurrently live events — were not characterised by the present investigation and remain to be established.

A third area is asynchronous programming via the `--!ndrone` syntax. The token has not been subjected to black-box probing; its precise semantics — whether it has any limitations or unexpected behaviors— are unknown to the present author. A focused investigation of `!--ndrone`, in the same experimental idiom as the present report, is a natural next step.

Further goals aim to study and reverse-engineer the mechanisms of Lau in order to exploit its features, flaws and limitations. The present report is the first step in a larger effort to understand the language and its hidden behind the scenes.

6 Parameters

This section enumerates the parameters held fixed and the parameters varied in the experiments reported above. Reproducing the findings requires controlling for these.

Lau version. All experiments were conducted against Lau version 5.4.1. Findings may not apply to other versions. The version was verified at the start of each session via the engine-reported version string.

Execution environment. All experiments were executed in the single-player sandbox. Behaviours that depend on multiplayer state, networking, or other players were not tested.

Timing method. Wall-clock timings were obtained from differences between successive `task.clock()` invocations. The function `task.clock()` was verified to return a monotonically increasing float with sub-millisecond resolution prior to the timing experiments.

Billing measurement. A reported billing cost (e.g. 0.07/stmt) is the wall-clock time consumed by a statement that performs no user-visible I/O. The reported values conflate billing cost and any incidental scheduling delays; the two were not separately quantified except in the `task.wait` experiments.

List-size cap. The 800-element cap was established by writing to `arr[801]` and observing the error `Assignment Error: The list is full! Maximum 800 elements.` The cap of 8000 stated in some documentation is contradicted by the engine's behaviour.

`task.wait` cost. The per-call cost of `task.wait` was measured by bracketing `task.clock()` calls and averaging across 100 iterations. The empirical model is `task.wait(N)` $\approx C + N$ seconds, where C is a per-call fixed overhead of roughly 0.08–0.13 seconds and N is the argument. The N-component is imprecise at sub-100 ms scale and warrants averaging over many calls; precise per-operation costs are not recoverable from single-shot measurements. There is no observable minimum-quantum floor: a one-shot measurement of `task.wait(0.001)` and `task.wait(0)` may both report approximately 0.2 seconds, but a 100-iteration average shows that the two values differ by approximately the value of N passed in.

type() taxonomy. The enumeration of type strings (`"Number"`, `"String"`, `"Boolean"`, `"null"`, `"List"`, `"Color"`) is the result of direct probing with one value of each observed kind. Functions and any other first-class types were not exhaustively tested.

Pattern matching. Pattern behaviour was tested with four representative patterns: `%d+`, `%w+`, a back-reference replacement `[%1]`, and a multi-capture match. The full Lua 5.1 pattern grammar was not exhaustively exercised.

Reproducibility note. Every test file referenced in this report is stored under `Lau/experiments/`. To reproduce an individual finding, locate the corresponding `1x`, `2x`, …, `6x` script in that directory, execute it through the Lau 5.4.1 interpreter, and compare the resulting output against the output quoted in the relevant section above.